Neon on Raspberry Pi 5 to accelerate RGB2GRay, 128bit (Q register) slower than 64bit(D register), why?

Asked 1 year, 11 months ago Modified 1 year, 11 months ago Viewed 858 times

0

As the title says, I have a Raspberry Pi 5 (Cortex-A76 by Armv8; four cores), and I use OpenCV to do something on it.

I use cv::cvtColor to get RGB2Gray, it is slow on Raspberry Pi 5, so I use neon_arm.h to optimize speed and it's indeed faster.

In Version-1, I use vld3_u8(), which loads a uint8x8x3_t of data from image each times (64 bit register).

In Version-2 I, use vld3q_u8() ,which load a uint8x16x3_t of data from image each times(128 bit).

I use std::chrono to timing the code; strangely, the Version-1 is faster than Version-2 (In my opinion should slower).

Version-1 : 10.5ms (average for 100times).

Version-2 : 12.7ms (avg for 100times).

The photo size is 3009x4248.

I don't know whether anything wrong (maybe device or code?).

Anyone can teach me why?

I show my code as follow .

Copy

```
void neon_cv_8(cv::Mat &src,cv::Mat &grey,std::size_t size)
{
    uint8_t *p_img = src.data;
    uint8_t *p_o = grey.data;
    uint8x8_t r_corr = vdup_n_u8(76);
    uint8x8_t g_corr = vdup_n_u8(150);
    uint8x8_t b_corr = vdup_n_u8(30);


    for(std::size_t i = 0; i<size ; i+=8)
    {
        uint8x8x3_t vdata = vld3_u8(p_img);
        uint16x8_t tmp = vmull_u8(b_corr, vdata.val[0]);
```

```
            tmp = vmlal_u8(tmp, g_corr, vdata.val[1]);

            tmp = vmlal_u8(tmp, r_corr, vdata.val[2]);
            uint8x8_t vgray = vshrn_n_u16(tmp, 8);

            vst1_u8(p_o, vgray);
            p_o += 8;
            p_img += 8 * 3;
        }
    }//version-1

    void neon_cv_16(cv::Mat &src,cv::Mat &grey,std::size_t size)
    {
        uint8_t *p_img = src.data;
        uint8_t *p_o = grey.data;
        uint16x4_t r_corr = vdup_n_u16(76);
        uint16x4_t g_corr = vdup_n_u16(150);
        uint16x4_t b_corr = vdup_n_u16(30);


        for(std::size_t i = 0; i<size ; i+=2*8)
        {

            uint8x16x3_t vdata = vld3q_u8(p_img);
            uint16x8_t v_b = vmovl_u8(vget_low_u8(vdata.val[0])),
                       v_g = vmovl_u8(vget_low_u8(vdata.val[1])),
                       v_r = vmovl_u8(vget_low_u8(vdata.val[2]));

            uint32x4_t tmp_low = vmull_u16(b_corr, vget_low_u16(v_b));
            uint32x4_t tmp_high = vmull_u16(b_corr,vget_high_u16(v_b));

            tmp_low = vmlal_u16(tmp_low,g_corr, vget_low_u16(v_g));
            tmp_high = vmlal_u16(tmp_high,g_corr,vget_high_u16(v_g));

            tmp_low = vmlal_u16(tmp_low,r_corr, vget_low_u16(v_r));
            tmp_high = vmlal_u16(tmp_high,r_corr,vget_high_u16(v_r));
            uint8x8_t vgray0 = vqmovn_u16(vcombine_u16(vrshrn_n_u32(tmp_low,8)
                            ,vrshrn_n_u32(tmp_high,8)));

            v_r = vmovl_u8(vget_high_u8(vdata.val[0])),
            v_g = vmovl_u8(vget_high_u8(vdata.val[1])),
            v_b = vmovl_u8(vget_high_u8(vdata.val[2]));

            tmp_low = vmull_u16(b_corr, vget_low_u16(v_b));
            tmp_high = vmull_u16(b_corr,vget_high_u16(v_b));

            tmp_low = vmlal_u16(tmp_low,g_corr, vget_low_u16(v_g));
            tmp_high = vmlal_u16(tmp_high,g_corr,vget_high_u16(v_g));

            tmp_low = vmlal_u16(tmp_low,r_corr, vget_low_u16(v_r));
            tmp_high = vmlal_u16(tmp_high,r_corr,vget_high_u16(v_r));
            uint8x8_t vgray1 = vqmovn_u16(vcombine_u16(vrshrn_n_u32(tmp_low,8)
                            ,vrshrn_n_u32(tmp_high,8)));

            vst1q_u8(p_o, vcombine_u8(vgray0,vgray1));
            p_o += 16;
            p_img += 48;
        }
    }//version-2
```

c++ gcc arm simd neon

Share Improve this question Follow

edited Jun 18, 2024 at 7:16

Christoph Rackwitz
16.6k 5 43 59

asked Jun 17, 2024 at 10:19

illusionaryshelter
1 1 2

2 What compiler? What optimization options? Does that CPU have anything like Turbo where it clocks higher until it gets too hot, then drops back to a sustainable speed? Does benchmarking the functions in the opposite order change the results? – Peter Cordes Jun 18, 2024 at 1:27

My optimization options are: -O3 -mcpu=cortex-a76 -free-vectorize and the version is Gcc-12.2.0 on Debian 12. I have installed the heat sink and fan on the CPU. It does indeed cause frequency reduction due to overheating but not when running code. I will change my benchmark later and try again. – illusionaryshelter Jun 18, 2024 at 2:56

hello, I opened the target file on IDA to look at the assembly, surprisingly the assembly code doesn't contain the neon instruct (just plain asm), but the value of __ARM_NEON macro is true. More generally, if I use the neon intrinsic function, it just generates plain asm code; if I embed the neon asm code then it doesn't generate any code in asm. I don't know what I should do now. – illusionaryshelter Jun 19, 2024 at 9:30

1 I don't think it's plausible that GCC could compile intrinsics to scalar asm on integer regs. Many AArch64 SIMD instructions use the same mnemonics as their scalar counterparts, e.g. `add`, but on vector registers which is how you can tell they're SIMD. (Unlike ARMv7 / AArch32 where the mnemonics were named like `vadd`.) If that doesn't explain what you're seeing, you could show an example of what you're talking about on godbolt.org (it has AArch64 compilers.) – Peter Cordes Jun 19, 2024 at 11:34

Everything is as you said. Thanks very much! – illusionaryshelter Jun 20, 2024 at 3:44

1 Answer

Sorted by: Highest score (default)

2

Unlike the old 32-bit ARMv7, ARM64 SIMD is 16 bytes wide. Intrinsics like vget_high_something are no longer free; they compile into shuffle instructions. vget_low_something are still free, though.

You should use `vmull_high_u16` and `vmlal_high_u16` to do the math on the higher halves of your vectors without shuffles. Similarly, `vqmovn_high_u16` to reduce result to bytes.

Update One more thing, it seems in your version 2 your math is too wide. You're upcasting bytes into `uint16_t`, then multiplying `uint16_t` into `uint32_t`, then compressing back into bytes. Try the following function instead, untested:

Copy

```
void neon_cv_v3( const cv::Mat& src, cv::Mat& grey, size_t size )
{
    const uint8_t* p_img = src.data;
    const uint8_t* const p_imgEnd = p_img + size * 3;
```

```
    uint8_t* p_o = grey.data;

    const uint8x16_t r_corr = vdupq_n_u8( 76 );
    const uint8x16_t g_corr = vdupq_n_u8( 150 );
    const uint8x16_t b_corr = vdupq_n_u8( 30 );

    while( p_img < p_imgEnd )
    {
        // Load 48 bytes = 16 BGR pixels, and split into channels
        const uint8x16x3_t vdata = vld3q_u8( p_img );
        p_img += 48;

        // Multiply/accumulate these numbers using widening multiplications
        uint16x8_t low = vmull_u8( vget_low_u8( vdata.val[ 0 ] ), vget_low_u8(
b_corr ) );
        uint16x8_t high = vmull_high_u8( vdata.val[ 0 ], b_corr );

        low = vmlal_u8( low, vget_low_u8( vdata.val[ 1 ] ), vget_low_u8( g_corr )
);
        high = vmlal_high_u8( high, vdata.val[ 1 ], g_corr );

        low = vmlal_u8( low, vget_low_u8( vdata.val[ 2 ] ), vget_low_u8( r_corr )
);
        high = vmlal_high_u8( high, vdata.val[ 2 ], r_corr );

        // Shift right by 8 bits using rounding, then truncate from uint16_t into
uint8_t
        const uint8x8_t grey8 = vrshrn_n_u16( low, 8 );
        const uint8x16_t grey16 = vrshrn_high_n_u16( grey8, high, 8 );

        // Store 16 bytes of the greyscale output
        vst1q_u8( p_o, grey16 );
        p_o += 16;
    }
} // neon_cv_v3
```

Share Improve this answer Follow

edited Jun 19, 2024 at 20:52

answered Jun 19, 2024 at 9:33

Soonts

22.4k 11 69 148

Sign up to request clarification or add additional context in comments.

Comments

Add a comment

Start asking to get answers

Find the answer to your question by asking.

Ask question

Explore related questions

c++ gcc arm simd neon

See similar questions with these tags.